

I Assignment: Low-Light Image Enhancement with Cross-Dataset Generalization

1 Task Overview

Low-light image enhancement is a classic computer vision problem with very practical consequences. In night photography, surveillance, robotics, and medical imaging, poor illumination can hide the information that matters most. A good enhancement model should do more than brighten an image: it should recover structure, preserve colors, and remain reliable when the input distribution changes.

In this assignment, the main challenge is not only to enhance images, but to do so in a way that generalizes across datasets. The model will be trained on one domain and then evaluated on another, so the real question is whether the learned representation captures the underlying illumination problem or only the quirks of a single dataset.



Figure 1: Example of low-light enhancement. The figure shows a severely underexposed input and the corresponding enhanced result.

2 Objective

The main goal of this project is to design, train, and evaluate a deep learning pipeline for low-light enhancement, with particular attention to cross-dataset generalization. You are expected to build a complete and reproducible workflow, from data loading and preprocessing to model training, evaluation, and analysis.

Sub-Objectives

The specific goals are to:

- acquire and preprocess a source dataset for paired low-light enhancement;
- implement a baseline encoder-decoder model, such as a compact UNet;
- explore at least one meaningful variant, for example an attention block, a Retinex-inspired branch, or a loss reweighting strategy;



- evaluate the model both in-domain and cross-dataset;
- analyze the failure modes and discuss why the model does or does not generalize.

3 Dataset

You are encouraged to use at least two publicly available datasets for this task. For example, LOL-v2¹ is a suitable choice for training, while LOL-v1² and ExDark³ can be used for evaluation and robustness testing.

The recommended protocol is to train on a source domain, validate in-domain, and then test both in-domain and on at least one cross-dataset scenario. Please make sure to check and report the license and terms of use for every dataset included in your experiments.

4 Data Preprocessing

All preprocessing steps should be clearly documented and justified in your report. Typical choices include resizing or cropping images to a manageable resolution, normalizing pixel values, and defining deterministic splits for training, validation, and testing. A resolution of 256×256 is usually a good starting point for most hardware setups.

Data augmentation techniques such as horizontal flipping, random cropping, or mild color jitter can help improve robustness. If you decide to include additional unpaired data for stress testing, describe precisely how it is used in the evaluation. It is important to avoid any overlap between training and test sets, especially in terms of scene or identity, to ensure a fair assessment of generalization.

5 Model Architecture

You are suggested to start from a lightweight encoder-decoder architecture, such as a compact UNet implemented in PyTorch or TensorFlow. The baseline should combine pixel-level and structure-level supervision, for example with a weighted sum of L1 and SSIM losses.

In addition to the baseline, you should explore at least one meaningful variant. This could involve architectural changes, such as adding an attention block or a Retinex-inspired branch, alternative loss weighting, or a data augmentation strategy designed specifically for low-light robustness. The focus should be on clarity and reproducibility: black-box solutions with no design choices or justification are discouraged.

6 Hints

You can:

- start from a stable and simple baseline before adding complexity;
- keep a small, fixed visual validation panel to inspect qualitative progress;
- report both full-reference and no-reference metrics when applicable;

¹<https://github.com/flyywh/CVPR-2020-Semi-Low-Light>

²<https://daooshee.github.io/BMVC2018website/>

³<https://github.com/cs-chan/Exclusively-Dark-Image-Dataset>



- inspect color cast, over-smoothing, residual noise, and halo artifacts separately;
- use mixed precision and early stopping to keep training time under control.

7 Training and Evaluation

Train the model on the training set using suitable losses and optimization techniques, and monitor the validation set to decide when to stop. You are free to choose the optimizer and schedule that best fit your architecture, but your choices should be explained in the report.

Recommended T4 setup:

- Resolution: 256×256 (fallback: 192×192)
- Batch size: 8-16
- Optimizer: Adam or AdamW
- Epochs: 80-120 with early stopping
- Mixed precision: recommended

Mandatory metrics:

- Full-reference: **PSNR**, **SSIM** where paired ground truth is available
- No-reference: at least one among **NIQE** and **BRISQUE**

For the final report, summarize the results in a clear table and include both quantitative and qualitative comparisons between the baseline and the variant. Always specify the hardware, random seed, and any other detail needed for reproducibility.

8 Model Analysis

A dedicated analysis section is mandatory. Do not limit yourself to reporting scores: explain what the model actually does and where it fails.

- Provide a failure taxonomy including at least color cast, halo artifacts, over-smoothing, hallucinated details, and residual noise.
- For each failure type, include visual examples and a short causal hypothesis.
- Discuss why cross-dataset performance differs from in-domain performance.
- Propose at least two concrete mitigation strategies and test at least one of them.

9 Deliverables

- A well-commented Python code implementation of the data preprocessing and model tuning pipeline.
- A report documenting the detailed approach, including data preprocessing, the model training, and evaluation results.
- Analysis and discussion of the model's performance, challenges faced, and recommendations for improvement.
- A presentation that will be discussed at the time of the Oral Exam.



**Politecnico
di Bari**

Deep Learning - Instructor: Vito Walter Anelli, Ph.D.
M.D. in Computer Engineering - Politecnico di Bari
Test code: 2026_III - Released on May 17th, 2026
Deadline: June 18th, 2026

Notes

You are encouraged to utilize existing deep learning libraries such as TensorFlow or PyTorch for implementing the model. Make sure to properly document your code and provide clear explanations in the report to demonstrate your understanding of the concepts and techniques used.